

Author	Mohamed Ali Chemseddine ROUABAH
Project Owner	Mohamed Ali Chemseddine ROUABAH (JR English)
Version	1.0
Date	September 3, 2026
Contact	kendanine8@gmail.com
Version History	Version 1.0 Date 2026-09-03 Author Mohamed Ali Chemseddine ROUABAH Description Initial complete specification

JR Booking Pro JR Booking Pro — Full Specification & Firebase Build Roadmap

This document is the working specification for JR Booking Pro, a scheduling and reservation platform on Firebase where professionals publish their availability and clients book time slots directly with them. It carries forward every feature from the original plan. Where the original plan contradicted itself, left a gap, or described something that is not technically safe or possible the way it was written, this version fixes it and explains the fix in place, and again in a summary at the very end so nothing has been changed silently. Nothing has been removed. The document is organized so it can be handed directly to a developer and built section by section, in the order given in Part 11, with the verification discipline in Part 12 ensuring each step is built one at a time and confirmed working by a human before the next one begins, so no new piece is ever added in a way that breaks something that already worked.

Part 1 Philosophy, Stack, and Language Standards

1.1 Development Philosophy

JR Booking Pro is built with a strategic focus on sustainability and financial efficiency, favoring tools that are free, reliable, and either open-source or carry a permanently generous free quota, so the platform stays accessible and scalable without proprietary overhead. In practice this means Firebase is used for hosting, authentication, database, and file storage because its free tier covers a new platform's needs and its services are tightly integrated, while every service that is not Firebase itself is chosen for being genuinely free or open-source wherever a real choice exists. The one deliberate exception is Google Calendar synchronization, which stays in the plan because it is an explicit, named feature the professionals need, not a default the platform imposes on itself; every other external dependency in the original plan is replaced with an open equivalent, described below.

For every file in the website's architecture, a companion documentation file exists explaining its purpose and behavior, kept in a [/docs](#) directory that mirrors the code's folder structure

exactly. This directory lives outside the Firebase Hosting public root so it documents the project without ever being served to a visitor. The convention and its exact layout are covered in Part 3.

The website's .env has the following:

```
GOOGLE_REDIRECT_URI=https://jr-booking-premium.web.app/api/calendar/google/callback  
JWT_SECRET=Jr_B00king_Pr3mium_S3cur3_T0k3n_2026! )
```

The website will be published on firebase on the following project:

```
https://console.firebase.google.com/project/jr-booking-premium/overview  
https://jr-booking-premium.web.app/
```

Branches for finished features are to be pushed to this repository:

```
https://github.com/Mohamed-Ali-Chemseddine-ROUABAH/JR-Booking.git
```

1.2 Language Standards

Naming conventions and all underlying code variable names, file names, function names, comments are written in English for technical clarity and consistency among developers. All user-facing interface text and website content are strictly in French. To make this maintainable rather than a source of scattered hardcoded strings, all French UI text lives in a small set of centralized string files (one per module, described in Part 3) rather than being written inline throughout the code, so a wording change never requires hunting through logic files to find it.

1.3 Technology Stack, Corrected and Clarified

The site is written in vanilla JavaScript, HTML, CSS classes, and Bootstrap only, exactly as specified. Each major function keeps its own file for the reasons already stated in the original plan: easier debugging, cleaner function boundaries, better readability, lighter loading for users who never touch a given feature, and cleaner modularity and security boundaries. Firebase provides Hosting for static delivery, Authentication for client and professional login (with custom claims used for the admin role), Cloud Firestore as the primary database, and Firebase Storage for the identity-verification documents uploaded during the professional account request — Storage was implied by that feature but never named outright in the original plan, and is added here because the upload has to live somewhere.

Two corrections are made to keep the stack consistent with the "free and open-source" principle. First, the original plan describes a map showing a professional's address, movement zones, and a calculated travel path to a client's address, which naturally implies Google Maps but the Google Maps JavaScript and Directions APIs are not free beyond a small quota and would conflict with the stated philosophy. Leaflet.js is used instead, a free, open-source mapping library, paired with OpenStreetMap tiles and a free routing source such as OpenRouteService's free tier for the distance and travel-time calculations. This

keeps the original requirement that route and travel-time computation happens locally in the client's browser rather than on the server, and the result is still saved to Firestore so it is never recalculated unnecessarily. Second, code splitting and lazy loading are achieved with native ES module dynamic `import()`, which lets the browser fetch a module say, the payment-info popup's logic only the moment a professional actually opens it, with no build tool required at all. If a bundling and minification pass is wanted later for production performance, Vite can be added as a free, open-source, dev-time-only tool without changing the modular file layout described in Part 3.

There is one piece of the original stack philosophy that needs a direct correction rather than a workaround: the plan says complex Cloud Functions should be skipped entirely and everything kept in standard client-side JavaScript. That holds true everywhere except three narrow, specific tasks that cannot be done safely, or at all, from a browser, because they require a secret or a trusted execution context the client must never hold. The first is sending transactional email a Gmail-based SMTP relay cannot be authenticated from client-side code without exposing credentials, so the platform instead uses Firebase's official **Trigger Email** extension, which needs no custom function code to be written or maintained, only a one-time configuration during setup, and is triggered simply by writing a document into a `mail` collection. The second is writing the platform's audit logs: if a client could write directly to the `logs` collection, a compromised or malicious client could forge or erase its own trail, so log entries are instead written by a small Cloud Function triggered automatically whenever a sensitive collection changes, using the Admin SDK, which bypasses client security rules entirely and therefore cannot be tampered with from the browser. The third is Google Calendar synchronization: the OAuth refresh token that keeps the sync alive must never be readable by client-side code, so token exchange, refresh, and the calendar webhook listener all live in a small, dedicated Cloud Function. These three are the only exceptions to the "no Cloud Functions" rule, they are each small and single-purpose, and everything else in the platform schedule rendering, booking logic, statistics, the client database, printing, exports runs entirely in the browser exactly as originally intended.

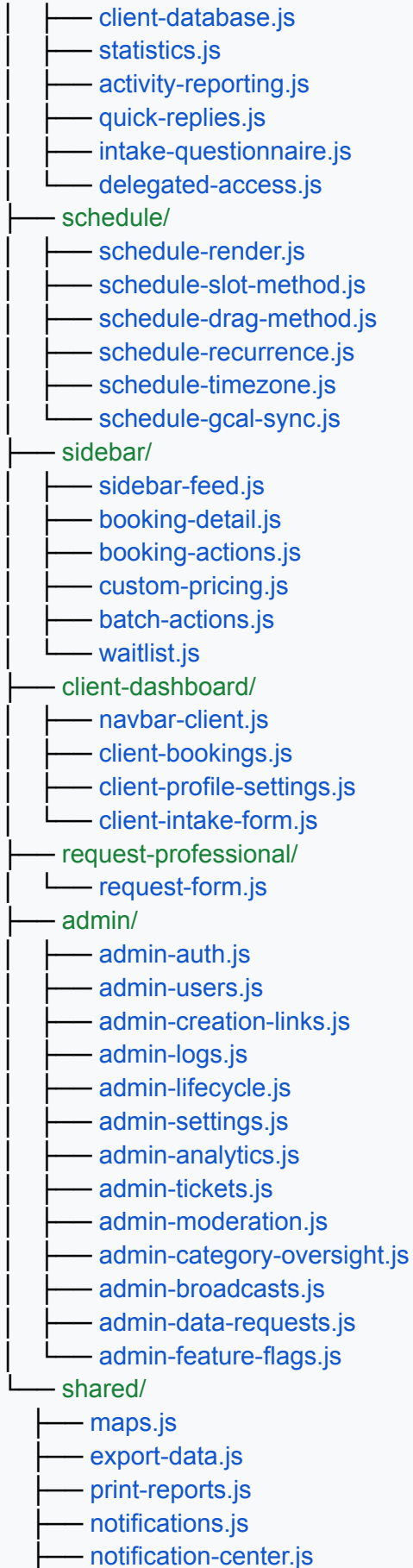
One practical consequence of these three exceptions is a required human setup step: Firebase's free Spark plan cannot make outbound network calls from Cloud Functions or Extensions, which both the email relay and the Calendar sync need, so the project must be upgraded to the Blaze (pay-as-you-go) plan. Under normal traffic this stays within Blaze's own free monthly quota and costs nothing, but the word "free" in the philosophy section should be understood as "free within a generous quota" rather than "impossible to ever be billed," and this upgrade step is called out explicitly in `SETUP_CHECKLIST.md` so it is never a surprise.

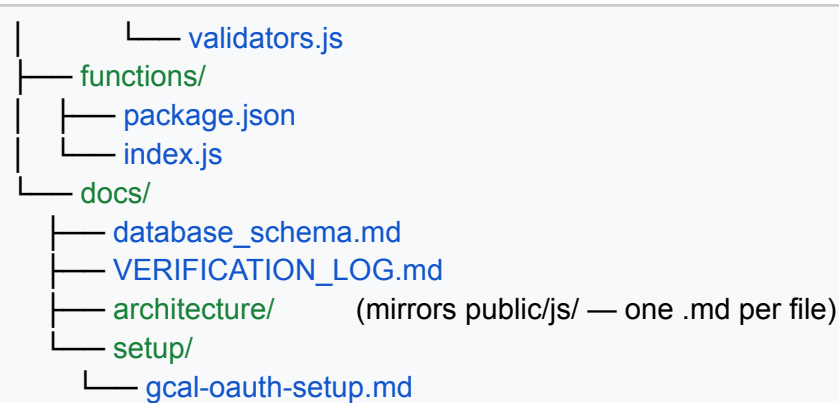
Part 2 Firebase Project and File Structure

The project is organized as a single Firebase project with the following layout. Everything under `public/` is what Firebase Hosting serves to visitors; everything outside it (`docs/`, `functions/`, the root configuration files) is either build-time tooling or internal documentation and is never deployed to the live site.

```
jr-booking-pro/
```

```
├── firebase.json
├── .firebaserc
├── firestore.rules
├── firestore.indexes.json
├── storage.rules
├── .env.example
├── SETUP_CHECKLIST.md
├── README.md
├── public/
│   ├── index.html
│   ├── login.html
│   ├── register-client.html
│   ├── request-professional.html
│   ├── profile.html
│   ├── category.html
│   ├── pro-dashboard.html
│   ├── client-dashboard.html
│   ├── admin-&lt;unlisted-token&gt;.html
│   ├── admin-dashboard.html
│   ├── assets/
│   │   ├── css/
│   │   │   ├── base.css
│   │   │   ├── glass-theme.css
│   │   │   ├── pro-dashboard.css
│   │   │   ├── client-dashboard.css
│   │   │   └── admin.css
│   │   └── img/
│   └── js/
│       ├── core/
│       │   ├── firebase-init.js
│       │   ├── auth-guard.js
│       │   ├── strings-fr.js
│       │   └── utils.js
│       ├── search/
│       │   ├── search-professional.js
│       │   └── search-category.js
│       ├── profile-public/
│       │   └── profile-view.js
│       ├── pro-dashboard/
│       │   ├── navbar.js
│       │   ├── working-hours.js
│       │   ├── payment-info.js
│       │   ├── work-type.js
│       │   ├── schedule-settings.js
│       │   ├── services-packages.js
│       │   ├── buffer-time.js
│       │   └── personal-info.js
```





Each HTML page is a shell that loads only the JS modules it needs, and each module loads its own sub-modules on demand through dynamic `import()`, so a professional who never opens the payment-info popup never downloads that code. The admin login page is deliberately not named anything predictable like `admin-login.html`; its actual filename should be a random, unguessable token chosen at deployment time, recorded in `.env` and `SETUP_CHECKLIST.md`, and never linked from any navigation element anywhere on the site this preserves the original plan's "separate page not linked anywhere" requirement while making the hidden name genuinely unguessable rather than just undiscoverable through the menu.

`functions/index.js` contains exactly the three narrow exceptions described in Part 1.3: the audit-log trigger, the Calendar OAuth handler, and the Calendar webhook receiver. Nothing else lives there, keeping the "skip complex Cloud Functions" principle intact everywhere else.

Part 3 Documentation and Human Setup Conventions

Every file under `public/js/` and every top-level HTML page has a matching Markdown file in `docs/architecture/`, at the same relative path, describing its purpose, which Firestore or Storage paths it reads and writes, which security rules it depends on, and which other modules it imports. This file is updated in the same commit or edit that changes the code it describes, so the documentation never drifts out of sync with what the code actually does. `docs/database_schema.md` is the single source of truth for every collection and field in Firestore and is appended to automatically whenever a new field is introduced, exactly as the original plan requires.

`SETUP_CHECKLIST.md` and `.env.example` guide the human operator through the minimal manual steps Firebase itself requires none of this can be automated away, since it involves clicking through Google's own console. The protocol assumes zero prior knowledge: every step is written in French and English side by side, every configuration value is named in both languages (for example, "Règles de base de données" / "Database Rules"), and every step links out to Firebase's own documentation or a video walkthrough for that exact screen. A representative pair of entries looks like this "1. Créez un projet Firebase sur console.firebase.google.com et activez l'authentification par e-mail/mot de passe. / 1. Create a Firebase project at console.firebase.google.com and enable email/password authentication. (Guide: <https://firebase.google.com/docs/auth/web/password-auth>)" and every step ends in a checkbox. Until a step is checked, the project is in a "Setup Pending" state for

anything that depends on it, and the developer working on the project proactively offers explanation, troubleshooting, and clarification rather than moving on, exactly as the original protocol specifies. The checklist should include, at minimum, initial project creation, enabling email/password authentication, upgrading to the Blaze plan, configuring the Trigger Email extension with a Gmail sending account, and setting up a Google Cloud OAuth client for Calendar sync, since all four are genuine external prerequisites nothing in the codebase can substitute for.

This same zero-knowledge, progress-gating pattern is not limited to the one-time environment setup above. Once building actually begins, every phase of the roadmap in Part 11 is verified by the human operator the same way, through a companion living file, [docs/VERIFICATION_LOG.md](#), before the next phase is allowed to start. The full mechanics of that ongoing, phase-by-phase verification discipline are described in Part 12.

Part 4 Database Architecture and Security Model

4.1 Flat Collections

"Flat" here means every collection sits at the top level and documents reference each other by ID rather than embedding one another wholesale, so a booking stores a professional's ID and a client's ID rather than a copy of their entire profiles. The handful of subcollections used below (a booking's own message thread, for instance) do not break this principle, since Firestore subcollections are never downloaded as part of their parent document and stay just as lightweight.

[proProfiles/{proId}](#) is the professional's private, full settings document: working hours, break rules, exceptions, vacation and absence periods, payment information, work-type toggles, address and movement zones, schedule-type settings, personal information, categories, experience, timezone, and a flag noting whether Calendar sync is active (never the tokens themselves). [publicProfiles/{proId}](#) is a denormalized copy containing only the fields the professional has explicitly toggled visible; it is what powers the front-page search, category browsing, and the public profile page, and it is the only professional-owned collection anonymous visitors can read at all. [clientAccounts/{clientId}](#) is a client's own login-linked account data. [proClientRecords/{proId}_{clientId}](#) is the professional-specific record about one client their custom rate, their movement-rate override, block or ban state, and notes kept separate from the client's own account because this is the professional's private annotation of that client, not data the client owns. [bookings/{bookingId}](#) holds the professional and client IDs (or embedded guest contact details if the professional created the booking for someone without an account yet), the start and end time, status, recurrence rule, an optional custom price override, who created it, and a [messages](#) subcollection for the reservation-linked conversation; reservation links (temporary or permanent call URLs) are stored as a small embedded array on the booking itself since there are only ever a handful per reservation. [creationLinks/{linkId}](#) holds the admin-issued professional onboarding tokens, their expiry, usage cap, remaining uses, and the accounts created through them. [logs/{logId}](#) is the tamper-proof audit trail, written only by the Cloud Function described earlier. [mail/{autoId}](#) is the Trigger Email extension's send queue. [gcalTokens/{proId}](#) holds Calendar refresh and access tokens and is never touched by client code at all.

One addition resolves a gap in the original plan: anonymous visitors need to see which time slots are already taken on a professional's public schedule, but they must never see who booked them, and a single Firestore document cannot expose different fields to different readers. A lightweight mirror, `busySlots/{proId}/{bookingId}`, is written automatically whenever a booking's status changes, containing only the start time, end time, and status nothing identifying. This is what powers the anonymous, read-only schedule view described in Part 6.

Supporting the additional professional and admin tools described in Parts 6.4 and 8, a handful of new collections and fields extend this schema without changing anything above. On `proProfiles`, new fields hold a buffer-time setting (minutes automatically reserved between two bookings), an array of named services or packages (each with its own duration and price), an array of saved quick-reply message templates, an intake-questionnaire configuration, and a `delegates` map naming any staff member permitted to help manage the schedule along with exactly which actions they're allowed to take. On `proClientRecords`, a `tags` array holds the professional's own custom labels for organizing a client beyond the fixed activity categories. `bookings` gains a `no-show` status value alongside pending, accepted, rejected, and done, an optional reference to which named service was booked, and a private `prepNotes` field visible only to the professional. A new `waitlistEntries/{proId}/{entryId}` collection holds clients waiting for a slot to free up on an otherwise full day, each entry storing the client's ID, the desired time window, and a `notified` flag. On the admin side, `supportTickets/{ticketId}` holds general platform questions or problems raised by any professional or client, tracked through the same Pending, In Progress, Completed states already used for professional applications; `dataRequests/{requestId}` tracks formal data-access or erasure requests for compliance purposes; `platformConfig/settings` is a single document holding feature toggles admins can flip platform-wide, such as temporarily pausing new professional registrations; and the `verified` flag that grants a professional's public profile its badge lives on `publicProfiles` but, unlike every other field there, is never written by the professional's own client code only an admin action can set it, which the security model below accounts for.

4.2 Statistics and Simple Aggregation

The original plan proposes skipping complex Cloud Functions and using Firebase batch writes directly from client code for simple statistics like total bookings, to keep the system lightweight. This is kept for low-stakes, cosmetic counters — booking counts and occupancy figures can be maintained with Firestore's `increment()` field value, restricted by a security rule that only allows the requesting professional to nudge specific counter fields on their own document by a delta, never to overwrite them outright. Revenue figures are different: because real money is represented, revenue should be computed by querying the actual "done" bookings and their resolved prices at read time rather than trusted from a client-writable running total, so a client can never quietly inflate their own revenue display. This keeps the "no Cloud Function needed" spirit of the original request while closing the one place it could otherwise be gamed.

4.3 Security Rules

Firestore rules default to denying all read and write access. Two deliberate, narrow exceptions are layered on top of that default rather than replacing it. Public read access is

granted only on `publicProfiles` and only for the fields the professional has toggled visible never the full `proProfiles` document, and never `busySlots` beyond start/end/status. Everywhere else, access follows an ownership check: a document is readable and writable only when the requesting user's ID matches the document's owner, or for professionals managing more than one profile appears in that document's `owners` array. Bookings use a two-sided version of this rule, since both a professional and a client have a legitimate stake in the same document: a client may create a pending booking or cancel their own pending request, but only the owning professional may accept, reject, mark it done, or set a custom price. This reconciles the original plan's "closed-network, deny by default" instruction with the fact that anonymous browsing, professional search, and category search all genuinely require some public read access the fix is not to weaken the default, but to be precise about the one denormalized, intentionally-public collection that carries it.

`storage.rules` governs the identity-verification documents uploaded during the professional account request: anyone submitting a request may write to their own request's folder (with rules validating file type and size), but only authenticated admins may read what's inside, and nothing there is ever public.

One clarification is worth stating plainly because the original plan leans on it implicitly: client-side validation and sanitization mentioned in the "Client-Side Standardization" section genuinely improves responsiveness and cuts down on obviously-bad writes, but it is never the actual security boundary, because a motivated user can always bypass the UI and call the Firestore API directly. The real enforcement layer is the security rules described above and, for the three narrow exceptions, the Admin SDK logic inside the Cloud Functions. Every client-side check in the interface should have a matching rule on the server side, so a direct, bypassed request is rejected exactly as a manipulated form would be.

Two of the newer additions need their own small carve-outs in the rules. Delegated staff access on a `proProfiles` document is not a simple owner match, since a delegate should be able to manage bookings without touching payment info, personal details, or the erase-account flow so the rule checks the `delegates` map for the requesting user's ID and the specific permission needed (for instance, `manageBookings`) rather than granting the blanket access the `owners` array gives a co-owning professional. The `verified` badge field on `publicProfiles`, and every field inside `platformConfig`, are both explicitly excluded from the professional's own write permission on those documents and can only be set through an admin-authenticated write, so a professional can never grant themselves a badge or flip a platform-wide switch by editing their own profile. `supportTickets` and `dataRequests` follow the familiar pattern readable and writable by whoever created them, and fully readable by any admin while `waitlistEntries` are readable and writable only by the client who joined the waitlist and the professional whose schedule they're waiting on.

Part 5 Navigation, Front Page, and Becoming a Professional

The front page carries the search bar for looking up a professional by name or category and the connection button that opens the professional login. The original plan also describes a large, round, prominent button that leads to the professional account request form, and separately says the front page holds "only" the search bar and the connection button — read

literally, those two statements conflict. The resolution used here keeps the front page visually minimal exactly as intended, with the round "become a professional" button present as a clearly secondary third element beneath or beside the search bar and connection button, so applying to become a professional is still reachable in a single click from the very first screen, honoring the three-click rule from the moment a visitor lands on the site.

Searching a professional's name takes the visitor to that professional's profile page, disambiguated by their unique `#id` tag when two professionals share a name. Searching a category was left incomplete in the original plan the sentence "If category name is searched:" was followed directly by the admin dashboard section with no description filled in. The behavior proposed here, flagged for confirmation in the summary at the end, is that a category search opens a results page listing every public profile with that category toggled on, shown as compact cards with the professional's name, id tag, and short description, each leading into the same profile page a direct name search would reach.

Part 6 Professional Dashboard

Once a professional logs in with their email and password, they land on their dashboard. The navbar shows their email, a printer icon for the reporting menu described in Part 9, a logout button, and a dropdown holding every settings section below.

The working-hours and vacation section opens a popup where the professional defines their working days and hours for each day of the week, break times attached to a given day either as a one-off exception or as a recurring rule, date-specific exceptions with their own hours, and vacation or absence periods spanning one or many days with a start time and an optional visible reason; a client viewing the schedule during an absence sees the start and end of the professional's working day, their break time, and their absence with its reason if one was given. This same section is also where the schedule's day-count view (from one to seven days, described in Part 7) and the professional's primary timezone are set, since both genuinely belong to "how my working time is defined" rather than anywhere else, and grouping them here keeps the original plan's intent while avoiding a stray setting living in an unrelated menu.

The payment-info section opens a popup with a toggle that, once activated, reveals fields to save French bank details; once saved, this information becomes visible on the client's own navbar dropdown for their active reservation. On the same popup, the professional can toggle on any of a list of accepted French banks, or add a bank name themselves along with a link, which is shown to clients marked as unverified since it was not vetted by the platform. Wero and similar phone-number-based payment options can be toggled on the same way. The professional also sets their rate per unit of time here. When a client clicks through to a payment option on their own dashboard, that click is tied to their specific ongoing reservation, and the client sees the balance owed at that moment or simply the reservation reference before being sent to the professional's chosen external payment page. The platform never processes a transaction itself; it only stores and securely surfaces the RIB or payment link the professional has provided, exactly as the original plan describes, and this is the one place in the spec where "private Firebase document" and "visible to the client" both apply at once by design, since the data lives in the professional's private `proProfiles`

document but is deliberately surfaced to a specific client only in the context of their own active booking, never broadcast publicly.

The work-type section opens a popup with independent toggles for online work and for movement (in-person travel to the client). Turning movement on requires saving an address; by default that exact address stays private and is used only internally to calculate distance and travel time, and a separate, explicit toggle must be switched on if the professional wants their exact address shown to clients this replaces a garbled sentence in the original plan with the behavior it appears to have intended. Once an address is set, a Leaflet map appears showing it, and beneath the map the professional can define movement zones as distance bands with an extra fee attached to each for example, zero to five kilometers at no extra charge, five to ten kilometers at an extra five euros and a client booking with a movement-enabled professional sees which zone they fall into and any resulting surcharge added to their running total. Next to the map, the professional picks their primary transportation method, with its own separate visibility toggle, so a client whose address has been entered can see both the estimated travel time and, if the professional chose to show it, how they're getting there; the professional sees the same calculated path and time in the other direction once a client's address is on file. As specified, this calculation always runs locally on each side using the free routing service described in Part 1.3, and only the result is saved and shared between the two.

The schedule-type section opens a popup where the professional chooses between two reservation methods: a client can click and drag across the schedule to define exactly what they want to reserve, or the professional can define fixed slots of a length they choose fifteen minutes, an hour, or any other duration that a client selects by clicking. The professional can allow or disallow selecting multiple slots at once, and cap how many, and can set a maximum number of times a recurring reservation is allowed to duplicate itself. On the same popup, recurring bookings can be permitted for a specific day of the week or permanently for all days, and the professional retains full control over any already-accepted recurring booking at any time they can reject a specific future occurrence, move its day or time, or shorten the remaining recurrence period. Slot or drag booking availability can likewise be set as permanent for the year, activated starting from a specific date and time, or limited to one specific day, entirely at the professional's discretion.

The personal-information section opens a dedicated page holding the professional's name, their unique id number (the one clients use to tell two same-named professionals apart), email, country, phone number, address, dashboard accent color, header and text color with a one-click reset to default, an optional student flag, links, a description, and their SIRET number. Each of these fields can be individually toggled visible or hidden on the client-facing profile. The professional can also choose up to ten categories from an existing list or create a new one a category the professional invents becomes searchable the moment it's saved, since category search simply queries the categories array already present on [publicProfiles](#) rather than needing a separate managed list and can add a CV-style list of past experience entries, each with a name and a year.

The client-database section opens a page listing every client the professional has ever booked with, filterable and classifiable by the parameters described in the original plan. Each row shows the client's email, booking name, contact info, phone, address, how many

bookings they've made, and their rate either the professional's standard rate or a rate set uniquely for that one client, including a per-client override of the movement-zone surcharge. A bin icon permanently erases a client from the database once the professional types a confirmation sentence; to keep this consistent with the other permanent-deletion flows described later, it follows the same thirty-day grace period and typed-confirmation pattern used for account erasure, implemented once as a shared utility rather than three separate copies of the same logic, which is a direct application of the platform's own modularity principle. An eye icon expands a list of the client's last fifteen actions, paginated for more, filterable by type or by date, showing every booking's full lifecycle from request to acceptance or rejection to completion. Next to it, a download icon exports that same history as a PDF for a chosen month or period. A block control locks the client out of booking with that one professional; a blocked client can only request to be unblocked, and the professional can accept, reject, or convert that into a permanent ban scoped to their profile alone, with no effect on any other professional. From the same page, a professional can flag a client to the platform's admin team requesting a full, site-wide permanent ban only an admin can actually execute that. An "export my data" action, using a native JavaScript function with no server round-trip, lets the professional download their own profile data locally as a JSON file at any time.

The statistics and analytics section presents real-time net and gross revenue accounting for completed and pending bookings, total bookings with completion and cancellation rates, an occupancy ratio of booked slots against total available working hours, and insight into recurring versus one-time clients to gauge retention. Data is shown through line charts for trends over time, pie charts for category distribution, and bar charts for side-by-side comparison, all filterable by date range, activity type, and booking status. Every metric and chart carries a hoverable "i" icon with a plain-language explanation of what it measures and how it's calculated. As noted in Part 4.2, revenue figures here are always computed from real "done" bookings and their resolved prices rather than a client-writable counter. The activity-reporting section builds on the same underlying data, generating a report of "done" classes, their prices, and calculated totals for whatever timeframe the professional selects.

6.1 Sidebar

The sidebar is collapsible and shows the feed of pending and accepted bookings; rejected bookings are hidden by default and surface only when the professional applies the rejected filter. Clicking a client's name in the feed expands that row downward, revealing accept and reject controls and a third control to modify the reservation opening a popup with the client's name, email, address, and every detail tied to their profile, along with the ability to change the booking's duration, time, or recurrence before it's accepted. Recurring bookings appear inside their slot in the feed with a checkbox to enable or disable that particular occurrence, and each occurrence's date and time can be individually reassigned. When two or more reservations overlap in time, they appear side by side in the feed as numbered chips in order of arrival; clicking one overlays it on the schedule so the professional can see exactly how the conflicting bookings sit against each other before deciding what to do. A message thread can be attached to any specific reservation and is visible to that client on their matching reservation for them to read and reply to. Temporary or permanent links most often video-call URLs can be attached to one reservation or to every reservation a given client makes; only the professional can add, edit, or remove them, and removing one takes it off both sides

immediately. Each booking slot carries a "done" checkbox, and this is the single source of truth every statistic and report in the dashboard relies on to know a class actually happened.

6.2 Schedule

The schedule shows every booking except rejected ones. Clicking an entry on the schedule highlights and scrolls to the matching row in the sidebar so an action can be taken on it. Double-clicking an empty slot opens a popup letting the professional create their own booking, either typing in details that don't exist anywhere in their client database yet, or selecting a previously entered client by email if that email has more than one guest profile attached (for instance, someone who has booked on behalf of several people), a numbered sub-selector lets the professional choose which one to auto-fill from. A booking created this way appears on the client's own dashboard in a distinct color and label marking it as professional-made; the client can accept it outright, or request a date change before accepting, which routes back to the professional for a final check, and the booking is only fully synced to both schedules once the client accepts.

The schedule defaults to a seven-day view, adjustable from one to seven days in the working-hours settings described above, and is navigated with vertical scrolling. Professionals have unrestricted access to both past and future dates; clients are limited to the current week for new or upcoming bookings, plus any past dates where they already have booking history of their own. A synchronization toggle in the corner of the schedule opens a secure Google account authorization popup; once granted, sync runs in both directions and in real time bookings made on the site push to Google Calendar and events created in Google Calendar pull into the schedule carrying contact details and the reservation's message thread along with the calendar event so a professional can see everything from either place. Imported Google Calendar events can appear in "ghost" mode, a non-interactive visual reference only, or "solid" mode, a fully integrated slot the professional can move and manage like any native booking; the original plan doesn't specify whether this choice is made once globally or per individual imported event, and the assumption used here noted again in the summary is a global default the professional sets, with the option to flip any single imported event to the other mode from its own right-click menu. Right-clicking any reservation opens a menu to mark it done, accept it, reject it, or return it to pending.

6.3 Responsive Behavior

Below a defined window-size breakpoint, the sidebar stops sitting beside the schedule and becomes a collapsible section stacked underneath it instead, so the layout stays usable for tablet and mobile visitors. Rather than being treated as a final pass bolted on after the desktop layout is finished, this breakpoint behavior should be built into each dashboard section as it's built in the first place using Bootstrap's grid, since retrofitting a two-column layout to collapse cleanly after the fact is far more error-prone than designing it responsively from the start; a dedicated QA pass at the very end of the build, described in Part 5's roadmap, exists only to verify this, not to introduce it for the first time.

6.4 Additional Professional Tools for a Smoother Client Workflow

None of the original plan's professional-facing features are removed or changed by what follows these are new additions, aimed squarely at the everyday friction of running a real

practice through the dashboard: knowing what needs attention right now, saying the same thing to clients less often, and keeping a full day of appointments from turning into a full day of manual admin work.

A notification bell sits in the navbar next to the printer icon, gathering everything that needs the professional's attention into one place: a new booking request, a new message on an existing reservation, a client's request to be unblocked, a waitlist match, or a support-ticket reply from the admin team rather than requiring the professional to dig through the sidebar to notice any of it. Right below the navbar, the dashboard's home view opens on a compact "today" widget showing the very next appointment and the full lineup for the day at a glance, before the professional has to scroll the full schedule to find it, which keeps the very first thing a professional sees each morning being exactly what they need to act on.

To cut down on repetitive typing, a professional can save quick-reply message templates: a reminder to arrive early, what to bring, a cancellation policy reminder and attach any of them to a reservation's message thread in one click instead of retyping the same note for every client. Pending bookings can also be acted on in batches: rather than opening each request one at a time, a professional can select several at once from the sidebar feed and accept or reject them together, which matters most for professionals who get a wave of requests at once. A configurable buffer time can be set in the schedule settings so a short gap—five minutes, fifteen, whatever the professional needs—is automatically held between two back-to-back bookings, without having to manually block it as a break every single day the way the original vacation and break system would otherwise require.

When a day is fully booked, a client can join a waitlist for it instead of simply finding nothing available; if an existing booking on that day is later cancelled or rejected, the platform notifies the next person on the waitlist automatically that a slot has opened, saving the professional from fielding "let me know if anything opens up" requests by hand. Alongside the existing accept, reject, and done states, a booking can also be marked as a no-show, which is distinct from a cancellation for statistics purposes and gives the professional an honest, trackable record of clients who don't show up, feeding into the client tags described below rather than requiring the professional to remember it themselves.

Rather than a single flat rate per unit of time, a professional can define named services or packages—for example, a fifteen-minute discovery call at no charge and a sixty-minute full session at a set price—and a client chooses which one they're booking rather than everything defaulting to the same generic rate; this sits alongside the existing rate and per-client override system rather than replacing it, since a per-client rate override can still apply on top of whichever service was chosen. In the client database, the professional can attach their own custom tags to a client: "VIP," "new," "needs a reminder call," or anything else that helps them organize at a glance separate from the fixed activity categories used for public search. From that same page, a professional can open a single unified view of every message they've ever exchanged with one specific client across all of that client's bookings, rather than having to reopen each past reservation individually to piece the conversation back together.

Two smaller additions round this out. A professional can generate a shareable direct link or a QR code built from it that takes a client straight to booking with them, or straight to booking

one specific service, skipping the front-page search entirely, which is genuinely useful when a professional wants to share their booking page on social media, in an email signature, or on a printed flyer. And a professional can define a short intake questionnaire that a new client is asked to fill out before their very first booking is confirmed, so the information a professional actually needs before a first session arrives up front instead of being chased down afterward through the message thread.

For professionals who work with an assistant or a front-desk person handling their calendar, delegated staff access lets the professional invite another person to help manage bookings — accepting, rejecting, and rescheduling — without handing over the full account. A delegate never sees payment information, personal details, or the ability to erase the account, and everything a delegate does is attributed to them individually in the same audit trail used everywhere else on the platform, so the professional always knows exactly who did what. Finally, each booking slot in the sidebar can carry a short private note — what to prepare, what was discussed last time, anything worth remembering — visible only to the professional and never shown to the client, giving them somewhere to keep session-to-session continuity without it needing to live in the client-facing message thread.

Part 7 Client Dashboard and Becoming a Professional

From the front page, a visitor searching a professional's name lands on that professional's public profile, showing every field the professional chose to make visible along with their timezone; the visitor can set their own local timezone, and the schedule display converts automatically to show every slot in their own time. A prominent, animated "reserve" button sits centrally on the page; clicking it opens a login page where the visitor creates an account with their email, sets a password, with a password-reset link available underneath, and is redirected to their own client dashboard once logged in. Before logging in, a visitor sees the professional's schedule in a read-only, anonymous mode: confirmed bookings are clearly marked as occupied and are completely non-interactive, so nothing can be accidentally selected or reserved without an account.

The client dashboard carries a navbar with the client's email, a logout button, a printer icon, and a dropdown to edit their own profile data and to review the payment details relevant to whichever professional they have an active reservation with. A sidebar lists the client's current bookings whether they made the request themselves or a professional created it on their behalf along with each one's status: pending, accepted, or rejected, and any professional-made booking awaiting the client's own confirmation appears clearly marked as such.

The portal to apply for professional status is the round button described in Part 5, reachable from the front page. It leads to a request form collecting the applicant's full details, an email confirmation step, and a secure upload of identity-verification documents to Firebase Storage. On submission, the data and files are transmitted automatically to the admin team, and the request's status is tracked as Pending, In Progress, or Completed, with a submission timestamp and, once review begins, an "in progress" timestamp. Admins review the documents and, on approval, the platform automatically sends a unique account-creation link that lets the applicant set up their confirmed account; the same link can optionally be

configured to let that one confirmed account manage multiple professional profiles, feeding directly into the multi-profile mechanism described in Part 8.

Part 8 Admin Dashboard

Admins log in through the unlisted, randomly-named page described in Part 2, known only to them and never linked from any navigation. Because a hidden URL is only an obscurity layer and not real protection on its own, the page itself still checks for a genuine authenticated session carrying an admin custom claim before rendering anything, so a guessed or leaked link still hits a real authorization wall rather than an open door.

Admins can ban any professional or client from their dashboard, requiring a typed confirmation sentence before the ban takes effect, and can reinstate an erased account with a single click that emails the account holder a reactivation link. Creating a professional account starts with the admin generating a creation link tied to a confirmed email, with a configurable expiry date and a usage cap on how many profiles that one link can create; the admin can see how many uses are left, how many profiles have been created through it, and the full history of who was created and when, all logged automatically. Admins can also help a locked-out user regain access by resetting their account, either wiping all linked data or issuing a fresh access code while keeping existing data intact.

A profile's lifecycle can be set by the admin: after a configured period of inactivity, a profile enters a limbo, deactivated state, after which a mandatory thirty-day grace period begins, requiring the same typed security-sentence confirmation used elsewhere in the platform, before permanent erasure. This is a distinct mechanism from a professional erasing one of their own clients or a professional erasing their own account, but all three now share the same underlying thirty-day-grace, typed-confirmation pattern, consolidated into one reusable module rather than three separate implementations, which is both a genuine fix and a direct expression of the platform's stated modularity principle.

Professionals managing multiple profiles under one confirmed account use a single Firebase Authentication identity — one email and password — that can own several [proProfiles](#) documents; switching between them after logging in once is a purely client-side selection of which owned document to load, never a second authentication step, which is exactly what "switching without needing to re-authenticate" requires and lines up cleanly with the owners-array rule described in Part 4.3.

Professionals managing multiple profiles under one confirmed account use a single Firebase Authentication identity one email and password that can own several [proProfiles](#) documents; switching between them after logging in once is a purely client-side selection of which owned document to load, never a second authentication step, which is exactly what "switching without needing to re-authenticate" requires and lines up cleanly with the owners-array rule described in Part 4.3.

The logs section is the platform's central audit repository. Each entry carries a timestamp, an event type such as account creation, ban triggered, reinstatement, or profile override, the acting admin's ID, the affected entity's ID, a success or failure outcome, and any relevant payload metadata such as a generated link hash or the specific configuration change made. Admins can filter by date range, by a specific admin or type of event, by a user's ID, email, or

a fragment of their name, and can isolate failed attempts or critical security triggers. The same interface tracks every generated creation link's full lifecycle when it was created, when it expires, and whether it's been redeemed and lets an admin jump from any log entry straight to the affected profile's current settings to correct something on the spot if needed.

Additional Administrative Capabilities

As with Part 6.4, nothing below changes or removes anything the admin dashboard already does these are new tools that were never named in the original plan, aimed at giving admins a fuller picture of the platform and less manual digging when something needs attention.

Logging in opens onto a command-center home view rather than a blank dashboard: a snapshot of pending professional applications, open support tickets, recent bans, and creation links about to expire, so an admin's very first look tells them what actually needs handling today. Alongside it, a platform-wide analytics and health overview gives a bird's-eye view the per-professional statistics in Part 6 can't total professionals and clients, overall booking volume and growth over time, which categories are most active together with the operational health of the platform's three narrow Cloud Function exceptions from Part 1.3: whether transactional emails are sending successfully, whether Calendar sync is erroring for any professional, and whether the audit-log writer is keeping up, so a quiet failure in one of those three places doesn't go unnoticed for days.

A general support-ticket inbox gives professionals and clients a way to reach the admin team about the platform itself — a bug, a question, something that doesn't fit the existing "flag a client for a platform ban" escalation — tracked through the same Pending, In Progress, and Completed states already used for professional applications, so the pattern stays consistent across the whole admin experience.

Because categories are created freely by professionals as they set up their profiles, near-duplicates are bound to appear over time "Anglais" and "English" describing the same thing, for instance so admins get a category-oversight tool to merge, rename, or remove categories platform-wide, keeping search and browsing useful as the number of professionals grows. Alongside it, lightweight content-moderation tools let an admin review and, if necessary, redact a reported profile description, photo, or message without needing to ban the entire account over one piece of content, reserving a full ban for when it's actually warranted.

Since identity-verification documents are already collected during the professional account request, admins can use them to grant a verified badge on a professional's public profile once their identity has been confirmed, giving clients an extra, admin-controlled signal of trust beyond whatever the professional says about themselves and, as noted in Part 4.3, this badge can only ever be set by an admin, never by the professional's own account.

For platform-wide communication a maintenance window, a new feature, a policy change admins can send a broadcast announcement to all professionals, all clients, or both, delivered through the same Trigger Email mechanism used for every other transactional message, so no separate email system needs to be built for it.

Given the platform's handling of personal and financial information, admins also have a dedicated place to log and fulfill formal data-protection requests a right-to-access or right-to-be-forgotten request arriving outside the normal in-app flows tracked from receipt through to fulfillment with its own audit entry, distinct from the standard account-erasure flow already described, since a formal compliance request needs its own paper trail.

Finally, two smaller tools round out the admin toolkit: professional applications can be reviewed and approved or denied in bulk rather than one at a time when several arrive together, and a set of platform-wide feature toggles — kept in the [platformConfig](#) document from Part 4.1 — let an admin pause new professional registrations or temporarily disable a specific feature like Calendar sync during an incident, without needing a code change or a redeploy to do it.

Part 9 Email, Printing, and Reporting

Every transactional email the platform sends creation links, password resets, booking confirmations, cancellations, and modifications, and a professional's own configurable reminder notices tied to their Calendar sync is delivered through the Trigger Email extension described in Part 1.3. The admin configures one Gmail-based sending account once, during setup, through the extension's own configuration screen; application code never touches the credential directly, it only writes a document describing the email to send into the [mail](#) collection, and the extension takes care of delivery and traceability.

A printer icon sits in the navbar of both the professional and client dashboards. Clicking it opens a menu with four options: printing the schedule, with an anonymous-mode toggle that redacts client names for privacy; printing monthly bookings; including prices; and a total summary report. All four are generated entirely client-side using a free, open-source PDF library, so no server component is needed for either printing or the per-client history export described in Part 6.

Part 10 Failsafe and Usability Standards

Across every role — professional, client, and admin — navigation and feature access are designed so any target feature or piece of information is reachable within three clicks, and every destructive or irreversible action, from banning a user to deleting a profile to canceling a booking, requires a confirmation prompt. Forms and dashboard edits auto-save in real time to prevent data loss, critical actions carry an undo or revert capability for immediate recovery from an accidental input, and any failure is met with a clear, non-technical explanation of what went wrong rather than a raw error. These standards are not a checklist reserved for the end of the project — each phase of the build roadmap below is expected to satisfy them for whatever it introduces as it's built, since adding undo and autosave to a feature after the fact is far more error-prone than building them in from the start. The final phase of the roadmap exists only to verify this has held true everywhere, not to introduce it for the first time.

Part 11 Phased Build Roadmap

This is the order the platform should actually be built in, so that each phase has everything it depends on already in place and nothing gets rebuilt because an earlier assumption

changed underneath it. Where two phases don't depend on each other, they're noted as safe to build in parallel by two different developers, which is exactly what the platform's modular file structure is meant to make possible. Every phase below is followed by the verification gate defined in Part 12 an automated self-check followed by a hands-on human test logged in [docs/VERIFICATION_LOG.md](#) and the next phase does not begin until that gate has been passed, which is what keeps this order from ever collapsing into several unfinished features being built at once.

Phase 0 Human environment setup. The Firebase project is created, email/password authentication, Firestore, Storage, and Hosting are enabled, the project is upgraded to the Blaze plan, the Firebase CLI is installed, and [.env.example](#) and [SETUP_CHECKLIST.md](#) are filled in. Nothing else in this roadmap can be built or even tested locally until this phase is checked off, so it is a hard gate and the first "Setup Pending" state the zero-knowledge protocol holds on.

Phase 1 Repository and shared scaffolding. The full folder structure from Part 2 is created, along with skeleton versions of [firebase.json](#), a default-deny [firestore.rules](#), [firestore.indexes.json](#), and [storage.rules](#); the base and glass-morphism CSS; [firebase-init.js](#); [strings-fr.js](#); [utils.js](#); and a working front page whose search bar is wired to an (initially empty) [publicProfiles](#) query. This establishes the modular pattern every later phase follows, so it has to exist first even though it does very little on its own yet.

Phase 2 Authentication core. Client self-registration with email, password, and a reset link is built, along with professional login and [auth-guard.js](#) for route protection. Professionals do not self-register for early development, one test professional account is seeded directly through the Firebase console or a short one-off local script, documented in [SETUP_CHECKLIST.md](#), so the professional dashboard can be built and tested in the phases that follow, well before the admin-driven onboarding flow exists later in the roadmap. This phase must come before any dashboard work, since every dashboard sits behind a login wall.

Phase 3 Security and infrastructure plumbing. [firestore.rules](#) is filled in with the public/private split and owner-array pattern from Part 4.3 and deployed early, then redeployed incrementally as new collections appear rather than being written all at once at the end. The Trigger Email extension is installed and configured, and the shape of a [mail](#) document is agreed on. The audit-log Cloud Function and a stub of the Calendar OAuth handler are scaffolded now, even though most of what they do stays unused until their feature phase arrives, because getting the Blaze-plan deployment pipeline working once, early, avoids debugging deployment issues at the same time as feature logic much later.

Phase 4 Professional dashboard shell. The navbar with its empty dropdown structure, the printer icon placeholder, logout, an empty sidebar feed, and an empty schedule grid are built. This locks in the responsive behavior from Part 6.3 sidebar beside the schedule versus stacked beneath it before any real feature fills the layout, which is far cheaper than reflowing a finished dashboard later.

Phase 5 Working hours, breaks, exceptions, vacation, day-count, and timezone. This has to exist before the schedule can render anything meaningful, since the schedule's shape

which hours are open, how many days show is derived entirely from this module's saved data.

Phase 6 Personal information, categories, experience, and the public-profile mirror.

This is the first phase that writes into `publicProfiles`, unblocking the front-page search, category browsing, and the public profile page. It should land before Phase 9's client dashboard, since a client needs something to find before there's anything for them to book.

Phase 7 Schedule rendering and booking creation. The calendar grid is built from Phase 5's settings, along with both reservation methods, their limits, the recurrence cap, the recurring-allowed and activation-window toggles, and anonymous read-only viewing through the `busySlots` mirror from Part 4.1. This is the first phase where a real booking document gets created, so everything downstream that touches bookings depends on it.

Phase 8 Sidebar booking management. Accept, reject, and modify controls, the expanding row detail, overlapping-reservation chips, messages, links, the "done" checkbox, and custom pricing are built here, depending entirely on Phase 7 producing real booking documents to manage.

Phase 9 Client dashboard. Login and registration already exist from Phase 2, so this phase adds the client-side navbar and dropdown, the booking sidebar and status view, the timezone control, and accepting or requesting a change to a professional-made booking. This can start as soon as Phase 7 exists and does not need to wait for Phase 8, making it a genuine candidate for a second developer to build in parallel.

Phase 10 Payment info and movement zones. Bank, RIB, Wero, and link fields, the per-time-unit rate, the Leaflet map, distance-band pricing, and the checkout-balance handoff on the client's side are built here. It comes after bookings exist because the "balance owed right now" figure it displays is computed from real booking and price data, but it doesn't depend on Phase 8's sidebar tooling, making it another candidate for parallel development.

Phase 11 Client database and CRM page. Filters, per-client rate and zone overrides, block/unblock/ban escalation, erase-with-confirmation, the fifteen-action history, and the per-client PDF export are built here, needing Phase 7 and 8's booking history to have anything to display and Phase 10's rate and zone concepts to override.

Phase 12 Statistics, analytics, and activity reporting. This needs a meaningful set of "done" bookings with resolved prices, so it naturally follows Phases 7, 8, and 10. It introduces no new data model every chart reads from booking data already established only new read-side views on top of it.

Phase 13 Professional account request and the full admin dashboard. The public request form with its Storage upload, and every admin capability ban and reinstate, creation-link management, password and access recovery, profile lifecycle and erasure, multi-profile appointment, oversight and override, appointing new admins, and the logs interface are built here, deliberately after the professional and client experiences are stable, since the admin tools mostly manage and audit those flows rather than create new ones of their own.

Phase 14 Google Calendar synchronization. The OAuth popup, ghost and solid modes, real-time bidirectional sync, and Calendar-linked reminder intervals are built last among the functional phases, because this is the most technically involved piece the only feature needing genuine external OAuth setup and a stateful server-side token store and because it writes directly into schedule and booking structures every earlier phase already depends on, so those need to be stable and well-tested first rather than chasing two moving targets at once.

Phase 15 Printing, export polish, and a documentation pass. The four print-menu options are finalized, the PDF export library is confirmed working everywhere it's used, and every file in `docs/architecture/` and `docs/database_schema.md` is checked against what actually shipped, since documentation drifts fastest right after several phases have landed in quick succession.

Phase 16 Final QA and deployment. `firestore.rules` is tested against the Firebase Local Emulator Suite under every role anonymous, client, professional, admin with deliberately adversarial requests to confirm the security model in Part 4.3 actually holds. Every module is checked against the three-click rule and the confirmation, autosave, undo, and graceful-error standards from Part 10. Module sizes and lazy-loading behavior are checked on a throttled connection. The project is then deployed with `firebase deploy`, exactly as the original philosophy specified from the start.

Fitting the New Additions Into the Roadmap

The professional and admin tools added in Parts 6.4 and 8 layer onto the roadmap above rather than requiring a separate pass through it. Buffer time and named services or packages belong with Phase 5, since both define what a booking even is before the schedule ever renders one. Quick-reply templates, batch accept and reject, and private prep notes extend Phase 8, once real booking and message data exists for them to act on; the waitlist and no-show tracking extend Phase 8 as well, since a no-show is really just another booking-action outcome and a waitlist only becomes meaningful once real bookings can fill up a day. Client tags and the unified per-client communication history extend Phase 11, alongside the rest of the CRM page. The shareable direct booking link extends Phase 6, since it's a public-profile feature, while the client intake questionnaire extends Phase 7, since it needs to appear at the exact moment a first booking is being created. Delegated staff access is best added once Phase 8's booking-management tools are stable, since a delegate's whole purpose is managing those tools, and it reuses the owner-array rule pattern from Phase 3 extended with the permission-scoped `delegates` map from Part 4.3. The notification center and the today-at-a-glance home view are cross-cutting both only become meaningful once the events they surface already exist so they're best added as a short pass immediately after Phase 8, revisiting the shell built in Phase 4.

On the admin side, the `platformConfig` feature-toggle document is worth pulling forward into Phase 3, since a kill switch is most useful if it exists before there's much to switch off. The verified badge, the support-ticket inbox, category oversight, content moderation, broadcast announcements, data-protection request handling, the platform analytics and health overview, and bulk application actions all extend Phase 13 alongside the rest of the admin dashboard, since each one manages or reports on flows that need to already exist and be stable before a tool for overseeing them is worth building.

Part 12 Build Discipline: Verifying Every Step Before Moving to the Next

Everything in Part 11 describes what gets built and in what order; this part describes the discipline that governs how each phase actually gets confirmed finished before the next one starts, so the platform never spends a day in a half-working state and a human never has to guess whether something is safe to build on top of yet. The same progress-gating principle already defined in Part 3 for the one-time Firebase environment setup nothing proceeds until the human operator explicitly confirms a step is done is extended here to cover every phase and every individual feature described in Parts 5 through 10, for the entire life of the build, not only its first day.

Each phase follows the same repeatable cycle regardless of which part of the platform it touches. The narrow slice of work described in that phase's paragraph in Part 11 is built on its own, touching only the files and schema fields Part 2 and Part 4 already assign to it. Before it is ever shown to a human, it passes a short self-check: the page loads with no console or deployment errors, the Firestore Local Emulator Suite confirms the security rules for any new or changed collection behave correctly for every relevant role anonymous, client, professional, delegate, and admin and the matching file in [docs/architecture/](#) has been written or updated, so documentation is never a phase behind the code it describes. Only once that self-check passes is the phase handed to the human for the second, decisive gate: a real, hands-on test of the actual feature in the actual deployed environment, described next.

[docs/VERIFICATION_LOG.md](#) is where that second gate lives, following the same bilingual, zero-knowledge format [SETUP_CHECKLIST.md](#) already established for environment setup. Every phase gets its own entry the moment it's ready for testing, written in French and English side by side: a plain description of what was just built, the exact page to open and the exact sequence of clicks needed to try it, written for someone with no technical background at all, and the result they should expect to see if it worked. Underneath, a pass, fail, or notes line is left for the human to fill in themselves, by hand, once they've actually tried it not a checkbox the system ticks on the human's behalf. Until that line says the phase passed, the project stays in the same "Setup Pending" state the original protocol already defines, the next phase does not begin, and the developer's role in the meantime is exactly what the original protocol already asks for: proactively offering to explain, troubleshoot, or clarify whatever didn't work as expected.

Because Part 2 keeps each feature in its own file and Part 4 keeps each feature's data in its own fields or collections, two phases should almost never need to touch the same file or the same piece of schema at all, which is what makes it possible to verify one phase without waiting on a later one to exist and without a later phase quietly undoing an earlier, already-confirmed one. On the rare occasion a later phase does extend a file an earlier phase already shipped Phase 8 adding batch actions to [sidebar-feed.js](#), which Phase 7 already relies on, is a good example the rule is that the change has to be additive: a new exported function, a new optional field, never a change to the behavior or signature of something a previously-passed phase already depends on. Security rules follow the same additive default, since every deploy in Part 11 layers a new, narrow allowance on top of the

existing default-deny base from Part 4.3 rather than loosening or replacing a rule the human already tested and confirmed.

A phase does not need the later phases that will eventually consume its output to already exist in order to be verified. The working-hours settings from Phase 5, for instance, can be confirmed correct by opening the Firebase console or the emulator's own Firestore viewer and checking that the saved document looks right, without needing the schedule-rendering feature from Phase 7 to exist yet to display it visually. This keeps early, foundational phases from silently waiting on much later ones just to be testable, which is what allows the roadmap to actually move at the pace Part 11 lays out rather than bottlenecking on whichever feature happens to be the most visually complete.

Unfinished work is also kept from ever being visible to a real visitor while it's still being built. The `platformConfig` feature-toggle document introduced in Part 4.1 for admin use serves a second purpose during the build itself: a feature actively under construction stays behind its own toggle, defaulted off for everyone, and is only switched on for the human tester's own account until its `VERIFICATION_LOG.md` entry is marked passed, at which point it's deliberately switched on for everyone. This means the already-working, already-confirmed parts of the site keep behaving exactly as they did the moment before a new phase started, for the entire time that phase is in progress, and a half-finished feature can never be stumbled into by a real client or professional.

Finally, every phase is committed to version control on its own the moment its `VERIFICATION_LOG.md` entry is marked passed, giving a clean, working point to return to if a later phase ever causes a problem. Firebase Hosting keeps its own history of every previous deployment and allows a one-click rollback to the last confirmed-working release if a deploy is ever found to have broken something after the fact, and `firestore.rules` changes are always tested in the Local Emulator Suite before being deployed, never pushed straight to a live project untested, so a bad rule is caught before it can lock out or expose real user data. Taken together, this is what makes the roadmap in Part 11 fast rather than slow: because each phase is small, additive, isolated behind its own toggle until it's confirmed, and testable without depending on work that hasn't happened yet, a human only ever has to verify the narrow slice that just changed, never the whole site over again, and progress never comes at the cost of something already working quietly breaking underneath it.

Part 13 Summary of Fixes, Clarifications, and Assumptions Made

This section exists so nothing above was changed silently. Each item names the original issue and the fix applied, for confirmation or correction.

Category search was left as an incomplete sentence in the original plan, cut off before describing what actually happens. The behavior proposed — a results page of matching profile cards leading into the normal profile page — is an assumption, not something recovered from the original text, and is the one addition in this document that isn't a fix to something written, but a fill for something missing.

The address-anonymity sentence in the work-type section was grammatically broken in the original ("kept anonymous if you wish no you must toggle another option"). It's rewritten here as: private and internal-use-only by default, with an explicit opt-in toggle to make it visible to clients — which appears to be the intended behavior, but is worth double-checking against what was meant.

The payment-info section describes data as living in a "private Firebase document" while also needing to appear on a client's dashboard — this isn't treated as a contradiction to resolve by hiding the data, but clarified as: private in that no one but the involved parties can see it, surfaced deliberately and only to a client with an active reservation.

The "skip complex Cloud Functions" rule and the "closed-network, deny by default" security rule both, taken completely literally, conflict with features the plan also asks for — Calendar sync, tamper-proof logs, transactional email, and anonymous browsing all genuinely require narrow exceptions. Those exceptions are named explicitly in Parts 1.3 and 4.3 rather than left as an implicit, unaddressed gap.

The implied use of Google Maps for the address, zone, and travel-time features conflicted with the "free and open-source tools" philosophy stated up front; Leaflet with OpenStreetMap and a free routing service is used instead, keeping the original requirement that the calculation happens client-side.

The admin login page being "not linked anywhere" was kept, but supplemented with a real authentication and authorization check on that page itself, since a hidden URL alone is not a security boundary if it's ever guessed or leaked.

The multi-profile professional mechanism wasn't fully specified in the original plan beyond "switch without re-authenticating." It's defined here as one Firebase Authentication identity owning several profile documents, switched between purely on the client side.

Client-side erasure of a client record, a professional's own account, and an admin-triggered profile lifecycle erasure were three separately described flows that all use the same thirty-day grace period and typed-confirmation pattern; they're consolidated into one shared mechanism here rather than three separate ones, in the spirit of the platform's own modularity principle.

The "Simplified Aggregation... batch writes directly from client code" instruction is kept for cosmetic counters like booking counts, but revenue figures are computed from real booking data at read time rather than a client-writable total, since a client-writable revenue counter is a genuine, avoidable security gap.

Firebase Storage was never named in the original plan despite the professional account request needing a file-upload utility for identity documents; it's added here as the obvious place that upload has to live.

The ghost-versus-solid mode choice for imported Google Calendar events wasn't specified as global or per-event; a global default with a per-event override was assumed here and is worth confirming.

The front page was described as holding "only" a search bar and a connection button, while a separate section describes a large round button leading to the professional request form with no other page named as its home. Both are kept on the front page, with the round button treated as a small, secondary third element rather than breaking the page's minimal design.

Copyright and Confidentiality

© 2026 Mohamed Ali Chemseddine ROUABAH (JR English). All rights reserved.

CONFIDENTIAL AND PROPRIETARY

This document contains confidential technical and business information belonging to Mohamed Ali Chemseddine ROUABAH. No part of this specification may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the owner.

Non-Disclosure Reminder

This specification is provided strictly for development, estimation, and internal review purposes. Unauthorized sharing, public display, or distribution to third parties is expressly prohibited and may result in legal action.